Modernização cloud-native da Loja Veloz

Relatório técnico teórico | Projeto Pedidos Veloz | 28 de outubro de 2026

Arquitetura de microsserviços e papel do DevOps

Microsserviços organizam um sistema como serviços pequenos, autônomos e alinhados a capacidades de negócio, comunicando-se por contratos de rede. No cenário proposto, Gateway, Pedidos, Pagamentos e Estoque são componentes separados; PostgreSQL persiste pedidos e saldo de estoque. Essa decomposição permite evolução e escala independentes, mas aumenta o número de falhas de rede, contratos, versões e dependências que precisam ser operados. Não é uma vantagem automática: limites de domínio, observabilidade e automação são condições para que o custo distribuído seja controlável.

DevOps é a prática cultural e técnica que aproxima desenvolvimento e operação por responsabilidade compartilhada, feedback curto e automação. O ciclo inclui revisão de código, testes, build reproduzível, publicação de artefatos identificáveis, implantação progressiva e observação do comportamento em produção. Métricas e incidentes devem orientar melhorias e rollback, em vez de depender de procedimentos manuais e conhecimento individual.

Conteinerização: Docker e Kubernetes

Uma imagem Docker empacota aplicação e dependências em camadas reproduzíveis; o contêiner executa esse pacote com isolamento de processos, rede e filesystem fornecido pelo sistema operacional. Docker/Compose é apropriado para construção, execução local e integração de múltiplos serviços com redes, volumes e variáveis num arquivo declarativo. Compose reduz divergência entre máquinas e oferece inicialização coordenada para desenvolvimento, mas não é, por si só, uma plataforma de orquestração distribuída com reconciliação de réplicas.

Kubernetes é a camada de orquestração. Deployments declaram estado desejado e fazem rollout/rollback de Pods; Services fornecem descoberta e balanceamento interno; ConfigMaps e Secrets separam configuração do artefato; probes controlam prontidão e recuperação; HPA ajusta réplicas segundo métricas. Docker e Kubernetes não são alternativas equivalentes: Compose padroniza a experiência local e Kubernetes opera cargas distribuídas em cluster.

Orquestração, resiliência e segurança

Orquestrar significa reconciliar continuamente estado observado e desejado, reagindo a falhas de processo/nó e mudanças de capacidade. A estratégia adotada é RollingUpdate com Pods prontos antes de remover réplicas antigas, suportada por readiness e liveness probes. Recursos requests/limits permitem escalonamento previsível. O HPA usa CPU como sinal inicial; isso exige Metrics Server e não substitui escalonamento de nós. Para cargas reais, calibrar limites e métricas com testes de carga e SLOs.

Configuração não sensível pertence ao ConfigMap; credenciais devem vir de Secret ou cofre externo. Secrets do Kubernetes são codificados em base64 e precisam de controle RBAC, criptografia em repouso e rotação. Pod Security Admission no nível *restricted*, usuário não-root, seccomp, filesystem somente leitura e remoção de capabilities reduzem privilégios. Rede externa deve terminar em TLS/WAF; o MVP não habilita service mesh, mTLS ou políticas de rede.

Entrega contínua e observabilidade

CI/CD em sistemas distribuídos

CI executa validações automatizadas em cada alteração; CD produz e implanta artefatos de forma repetível, limitando risco por gates e rollout. Neste trabalho, PRs executam checagem de sintaxe e testes; merges na branch principal constroem e publicam imagens no GHCR com tag imutável do SHA. Uma execução manual e protegida do ambiente production aplica os manifests e aguarda o rollout. Credenciais de cluster e banco são secrets do ambiente, não arquivos do repositório. Em maturidade posterior, assinatura/verificação de imagem, scan de vulnerabilidades, promoção entre ambientes e aprovação por SLO completam os gates.

Versionar imagens pelo commit conecta o que foi testado ao que foi implantado e permite rollback rastreável. Cache do BuildKit reduz tempo sem alterar o conteúdo lógico do build. O pipeline separa permissões: leitura do código para testes e publicação somente quando a branch é autorizada; segredos de implantação não são necessários nos PRs.

Observabilidade: métricas, logs e traces

Observabilidade combina sinais complementares. Métricas agregam comportamento ao longo do tempo, como taxa de requisições, latência, erros, saturação de CPU/memória e conexões do banco; Prometheus coleta as métricas HTTP expostas por cada API em `/metrics`. Logs estruturados em stdout/stderr registram eventos e erros para agregação centralizada. Traces correlacionam spans de uma requisição entre gateway, serviços e dependências, identificando latência e falha no caminho completo.

As imagens iniciam as APIs com o agente `opentelemetry-instrument`, que instrumenta Flask, chamadas HTTP via requests e consultas psycopg. O contexto W3C Trace Context atravessa o gateway e os serviços; os spans seguem por OTLP para um OpenTelemetry Collector e são consultáveis no Jaeger. Prometheus é provisionado por Compose e Kubernetes para coletar as métricas HTTP. OpenTelemetry padroniza geração/exportação, não é armazenamento nem interface de consulta.

Justificativa das decisões

> **Compose local:** ambiente com um único comando, Postgres persistente e healthchecks, adequado ao prazo de quatro semanas.
>
> **Kubernetes:** Deployments, Services, probes e HPA tornam estado e rollout declarativos; PostgreSQL StatefulSet serve como demonstração, não como banco de produção resiliente.
>
> **Rolling + HPA:** estratégia simples de baixa indisponibilidade, com rollback por revisão; mínimo de duas réplicas e meta inicial de CPU a 70%.
>
> **Sem mensageria/mesh:** as operações demonstradas são síncronas e o MVP não necessita de brokers nem gestão de tráfego avançada; adotá-los sem requisito aumentaria custo operacional. Estoque e pedidos persistem em banco compartilhado no protótipo, decisão que deve evoluir para isolamento de dados e eventos/outbox em escala real.

Referências primárias e exemplo de referência

- Kubernetes: Deployments, probes, HPA e Pod Security Standards.
- Docker Compose: docs.docker.com/compose; boas práticas de build: Docker Docs.
- GitHub Actions: docs.github.com/actions; Terraform EKS: módulo oficial no Registry.
- OpenTelemetry: opentelemetry.io/docs/what-is-opentelemetry; 12-Factor: 12factor.net.
- Exemplo concreto: Google Cloud Online Boutique, repositório público com e-commerce demonstrativo de 11 microsserviços executável em Kubernetes e documentação de arquitetura. É referência técnica, não alegação de case de produção.